pfi

# Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

# pandas: Series and DataFrames

WEEK 2 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# By the end of today you can

## Load a holdings file

Its size, its columns, their types, what is missing.

```
holdings = pd.read_csv(data_path)
```

## Look up a bond

Columns by name, and a bond by its CUSIP.

```
by_cusip.loc['99039NAC0', 'price']
```

## Filter, count, and rank

By sector, rating, price, spread, or duration.

```
holdings[holdings['sector'] == 'Energy']
```

## Add a column, run a what-if

On a copy, without touching the desk's record.

```
what_if = by_cusip.copy()
```

# From four bonds to two hundred

Week 1

**4**

bonds, 6 columns

→

Today

**200**

bonds, 19 columns

Each bond now comes with text (white) and numbers (dark)

| | | |
|---|---|---|
| CUSIP | Issuer name | Sector |
| Rating | Maturity date | Price |
| Spread | Duration | Position size |

The tools so far, and the one for today

## A list or a dictionary

Holds the data. Answers no question without a loop.

→

## A NumPy array

Calculates on a whole column at once. Holds one type of value, and knows nothing about tickers or column names.

→

## A pandas table

Text and numbers side by side. Every column with a name. Every row a bond.

**Series: one labelled column. DataFrame: a table of them.**

# A Series is one labelled column

**prices**

| index | values |
|---|---|
| QVNE | 98.50 |
| DNMR | 103.25 |
| PLWK | 95.00 |
| HLVF | 101.00 |

dtype: float64

Built from two lists. Looked up by label, or by position.

```
prices = pd.Series(price_list, index=ticker_list)

print('PLWK:', prices['PLWK'])
PLWK: 95.0
print('First price:', prices.iloc[0])
First price: 98.5
```

## Two arrays in different orders

Multiplied position by position. No error, and four wrong numbers.

## Two Series in different orders

Lined up by ticker first, as a VLOOKUP would. A ticker missing on one side gives NaN, with no error.

# A DataFrame is a table of Series

desk: four rows, six columns, shape (4, 6)

| | Ticker | Coupon | Maturity | Price | Face | Rating |
|---|---|---|---|---|---|---|
| 0 | QVNE | 5.250 | 2033 | 98.50 | 1500000 | BBB |
| 1 | DNMR | 6.125 | 2034 | 103.25 | 500000 | BB+ |
| 2 | PLWK | 4.750 | 2031 | 95.00 | 750000 | A- |
| 3 | HLVF | 7.500 | 2032 | 101.00 | 250000 | B+ |
| **dtype** | str | float64 | int64 | float64 | int64 | str |

**Each column has one type, like an array. The table as a whole can mix them.**

Every column is a Series

```
desk_prices = desk['Price']
print(type(desk_prices))
<class 'pandas.Series'>
```

## In Excel

A worksheet with a header row.

The column names are the headers.

The index is the row numbers down the left edge.

# The holdings file: 200 bonds, 19 columns

```
holdings = pd.read_csv(data_path)
```

One row per bond. The first line of the file becomes the column names. Eight of the nineteen columns are shown.

| | cusip | ticker | sector | rating | coupon | price | oas | par_held |
|---|---|---|---|---|---|---|---|---|
| 0 | 99080CAE8 | BEZE | Financials | BBB- | 7.000 | 106.727 | 147 | 5000000 |
| 1 | 99049XAE2 | BEZH | Industrials | B | 8.125 | 97.556 | 535 | 250000 |
| 2 | 99039NAC0 | BEZL | Transportation | BB | 5.125 | 98.730 | 153 | 1500000 |
| **units** | text | text | text | text | percent of face | per 100 of face | basis points | dollars of face |

### `holdings.shape`

(200, 19): 200 bonds, 19 columns.

### `holdings.info()`

200 non-null in every column. Nothing is missing.

```
holdings['oas']
# one name: a Series
holdings[['ticker', 'coupon', 'maturity', 'price']]
# a list of names: a smaller table
```

# Rows: iloc by position, loc by label

```
by_cusip = holdings.set_index('cusip')
```

| position | cusip (the index) | ticker | coupon | price |
|---|---|---|---|---|
| 0 | 99080CAE8 | BEZE | 7.000 | 106.727 |
| 1 | 99049XAE2 | BEZH | 8.125 | 97.556 |
| 2 | 99039NAC0 | BEZL | 5.125 | 98.730 |

*Sort the table or remove a bond, and position 0 is a different bond. A label stays with its row.*

**`iloc`** **By position. A slice leaves out its end.**

`by_cusip.iloc[0:3]` positions 0, 1, 2

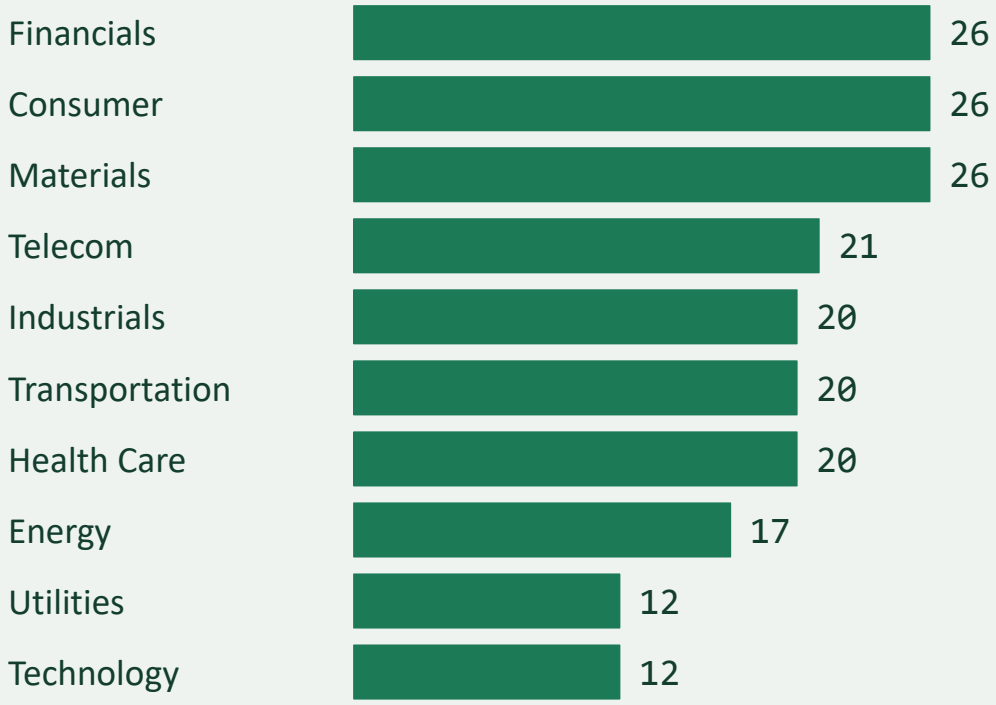
**`loc`** **By label. A slice includes its end.**

`by_cusip.loc['99080CAE8':'99039NAC0']` both ends

| Question | pandas | Excel |
|---|---|---|
| Price of a CUSIP | `by_cusip.loc['99039NAC0', 'price']` | `=VLOOKUP("99039NAC0", B:I, 8, FALSE)` |
| A CUSIP that is not there | `KeyError` | `#N/A` |

# A column is a Series, so it can be summarized

**`holdings['sector'].value_counts()`**




*A count of an invented portfolio. In Excel, a pivot table.*

The same job as the Excel function, with the column named

| pandas | Excel |
|---|---|
| `holdings['market_value'].sum()` | `=SUM(S2:S201)` |
| `holdings['oas'].mean()` | `=AVERAGE(M2:M201)` |
| `holdings['oas'].median()` | `=MEDIAN(M2:M201)` |
| `holdings['price'].min()` | `=MIN(I2:I201)` |

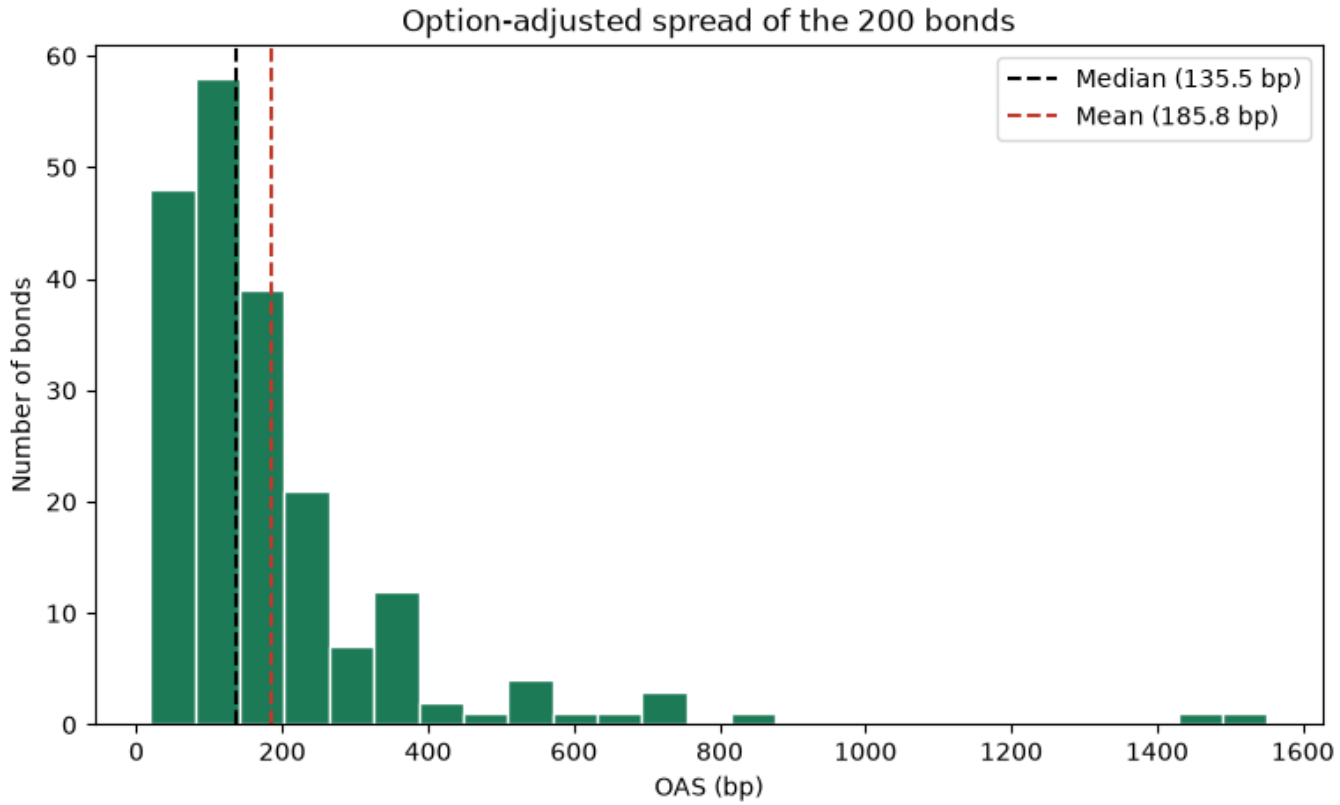
**115** `nunique()`

tickers. Many issuers have more than one bond.

**200** `nunique()`

CUSIPs for 200 rows. Every label is different.

# Mean and median, one picture




### Most bonds are tight

A tall block of bonds at spreads under 200 bp

### A few are very wide

The tail runs out past 1,400 bp

### The tail pulls the mean

Mean 185.8 bp, median 135.5 bp. Quote both.

*Read the picture and check it against the numbers in the notebook.*

# A filter is a column of True and False

**`holdings`**

| | ticker | sector |
|---|---|---|
| 0 | BEZE | Financials |
| 1 | BEZH | Industrials |
| … | … | … |
| 198 | ZEPN | Energy |
| 199 | ZEPN | Energy |

the mask

| is_energy |
|---|
| False |
| False |
| ... |
| True |
| True |

**`holdings[is_energy]`**

| | ticker | sector |
|---|---|---|
| 67 | GALN | Energy |
| 68 | GALN | Energy |
| … | … | … |
| 198 | ZEPN | Energy |
| 199 | ZEPN | Energy |

**17**

Energy bonds: the mask, summed

**38,830,005**

Their total market value

| Question | pandas | Excel |
|---|---|---|
| Show the Energy bonds | `holdings[holdings['sector'] == 'Energy']` | AutoFilter on the sector column |
| Count the Energy bonds | `(holdings['sector'] == 'Energy').sum()` | =COUNTIF(E:E, "Energy") |
| Market value of the Energy bonds | `holdings.loc[is_energy, 'market_value'].sum()` | =SUMIF(E:E, "Energy", S:S) |

# & narrows. | widens.

### & keeps the rows where both conditions are True

```
long_and_low = (holdings['duration'] > 10) & (holdings['price'] < 90)
print('Count:', long_and_low.sum())
Count: 6
```

**6** bonds pass both tests

### | keeps the rows where at least one condition is True

```
low_or_wide = (holdings['price'] < 80) | (holdings['oas'] > 1000)
print('Count:', low_or_wide.sum())
Count: 4
```

**4** bonds pass at least one test

**`( ... ) & ( ... )`**

Each condition in its own round brackets.

**`and, or`**

Same ValueError as in NumPy. Write & and |.

**`isin(['A+', 'A', 'A-'])`**

True where the value is in the list. 56 single-A bonds.

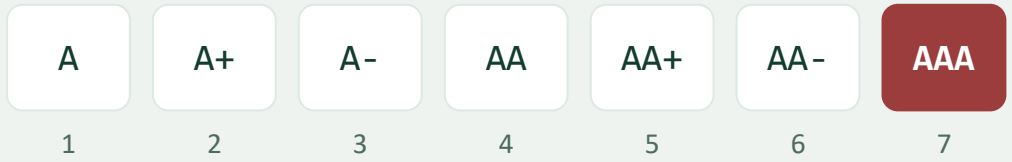
# Sorting hands back a sorted copy

```
widest = holdings.sort_values('oas', ascending=False)
```

| | ticker | sector | rating | price | oas |
|---|---|---|---|---|---|
| 65 | FESR | Technology | CCC+ | 87.116 | 1551 |
| 17 | CLDB | Consumer | B | 62.045 | 1446 |
| 132 | PRAN | Financials | CCC+ | 89.407 | 871 |
| 110 | NEVN | Utilities | B- | 96.616 | 720 |
| 111 | NEVN | Utilities | B- | 94.053 | 719 |

*The labels moved with their rows: the first row is still labelled 65. holdings itself is not sorted. The issuers are invented.*

Sort by rating, and text sorts letter by letter




**Alphabetical order, not credit order. AAA comes out seventh.**

```
holdings.nlargest(5, 'market_value')
```

A shortcut for sort, then head.

## In Excel: Data, Sort

Excel sorts the sheet in place. pandas leaves the original order alone.

# A new column is a formula column

```
holdings['coupon_income'] = holdings['par_held'] * holdings['coupon'] / 100
```

| | ticker | coupon | par_held | coupon_income |
|---|---|---|---|---|
| 0 | BEZE | 7.000 | 5000000 | 350000.0 |
| 1 | BEZH | 8.125 | 250000 | 20312.5 |
| 2 | BEZL | 5.125 | 1500000 | 76875.0 |

## In Excel

`=R2*G2/100`

A formula in a new column, filled down. A pandas column holds values, not live formulas: change an input, and rerun the line.

### 1 Work on a copy

```
what_if = by_cusip.copy()
```

### 2 Set one value with loc

```
what_if.loc['99039NAC0',
            'price'] = 97.0
```

### 3 Recalculate, then compare

```
print('Change in market value:',
      round(change, 2))
Change in market value: -25950.0
```

# Change many rows at once, on a copy

### loc[mask, column] = value sets many rows

```
marks = holdings.copy()
marks['price_check'] = 'OK'
marks.loc[marks['price'] < 80,
          'price_check'] = 'Confirm'
Rows marked Confirm: 3 of 200
```

CLDB at 62.045, NEVX at 73.873, SYLX at 76.903. The other 197 rows still read 'OK'.

### Filter, then .copy(), before you change the table

```
energy_what_if = holdings[is_energy].copy()
energy_what_if['price'] = energy_what_if['price'] - 2
Average Energy price in the what-if:  95.407
Average Energy price in the holdings: 97.407
```

The holdings did not move. With .copy() the code gives the same result in every version of pandas.

| Step | pandas | Excel |
|---|---|---|
| Set where a test holds | `marks.loc[marks['price'] < 80, 'price_check'] = 'Confirm'` | =IF(I2<80, "Confirm", "OK") filled down |
| Copy the filtered rows | `holdings[is_energy].copy()` | AutoFilter, copy the rows shown, paste |

# .style formats a table for display

**Format, title, shade, and bar. The numbers underneath do not change.**

```
(top10.style
  .format({'price': '{:.2f}',
           'oas': '{:,.0f} bp'})
  .background_gradient(
      subset=['oas'], cmap='Greens')
  .highlight_max(subset=['price'])
  .set_caption('Ten widest spreads'))
```

A cell still holds 106.727 when Format Cells shows 106.73. Style changes the display only.

Each method has an Excel twin

| pandas | Excel |
|---|---|
| `.format()` | Format Cells |
| `.background_gradient()` | Conditional Formatting, Color Scales |
| `.bar()` | Conditional Formatting, Data Bars |
| `.highlight_max()` | Top/Bottom Rules, Top 1 |
| `.set_caption()` | a title above the table |

# The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.

1

## Read

Read the code before you run it.

2

## Predict

Decide what it should produce.

3

## Run

Run it and look at the output.

4

## Check

Compare with a number you worked out yourself.

*Code that runs is not the same as code that is right.*

# What you can do now

- Load a holdings file and take a first look at its size, columns, and types
- Pull out the columns a question needs, and look a bond up by its CUSIP
- Filter by sector, rating, price, spread, or duration, and rank what passes
- Add a column, change rows where a test holds, and run a what-if on a copy
- Build tables, read shares, and show a table with formats, shading, and bars

**Tomorrow** pandas: joining tables, reading and saving files, summaries by group, and dates